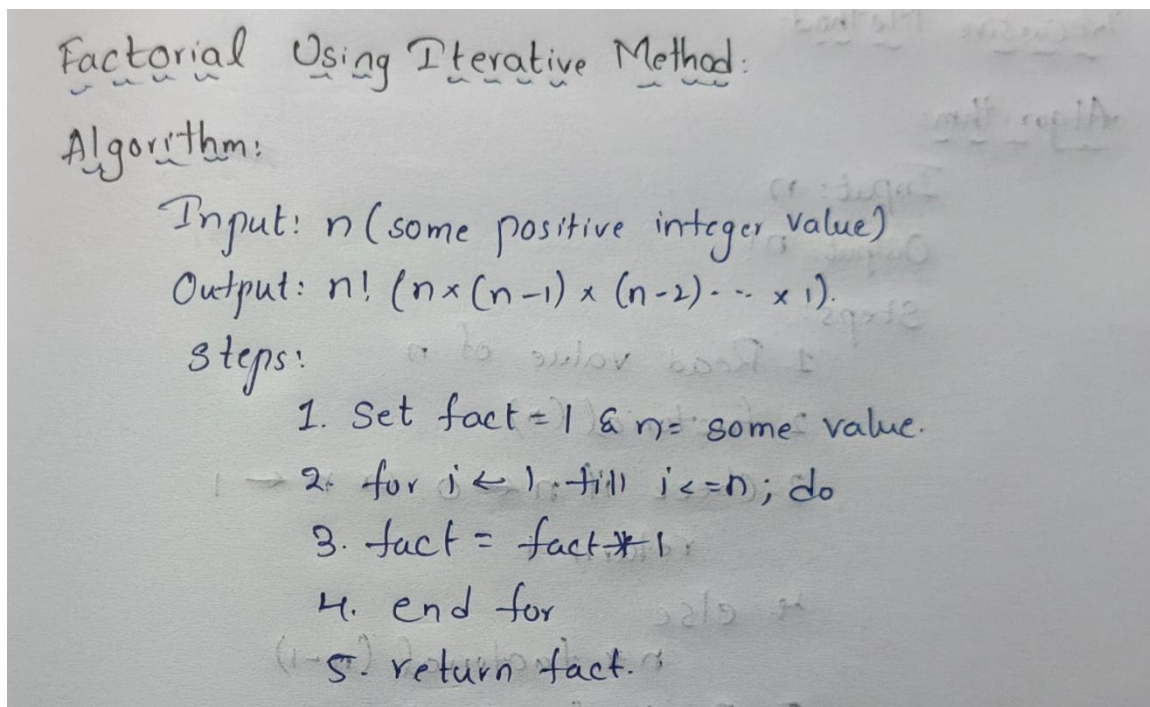


Practicle-4

Aim: Implementation and Time analysis of factorial program using iterative and recursive method

Iterative Algorithm:



Code:

```
#include <iostream> using
namespace std;

int main()
{
    int n; int fact = 1; cout << "Enter a
    positive integer: "; cin >> n;

    for (int i = 1; i <= n; i++)
```



```
{  
  
    fact = fact * i;  
}  
  
cout << "Factorial of " << n << " = " << fact;  
  
return 0;  
}
```

Output:

```
PS C:\Users\prash\Downloads\DAA Lab> cd "c:\Users\prash\Downloads\DAA Lab\" ; if  
($?) { g++ FactorialusingIterativeMethod.cpp -o FactorialusingIterativeMethod }  
; if ($?) { .\FactorialusingIterativeMethod }  
Enter a number: 5  
Factorial = 120  
PS C:\Users\prash\Downloads\DAA Lab>
```

Time complexity:

Time Complexity:

```
for (int i=1 ; i<=n; i++) {  
    fact = fact * i;  
}
```

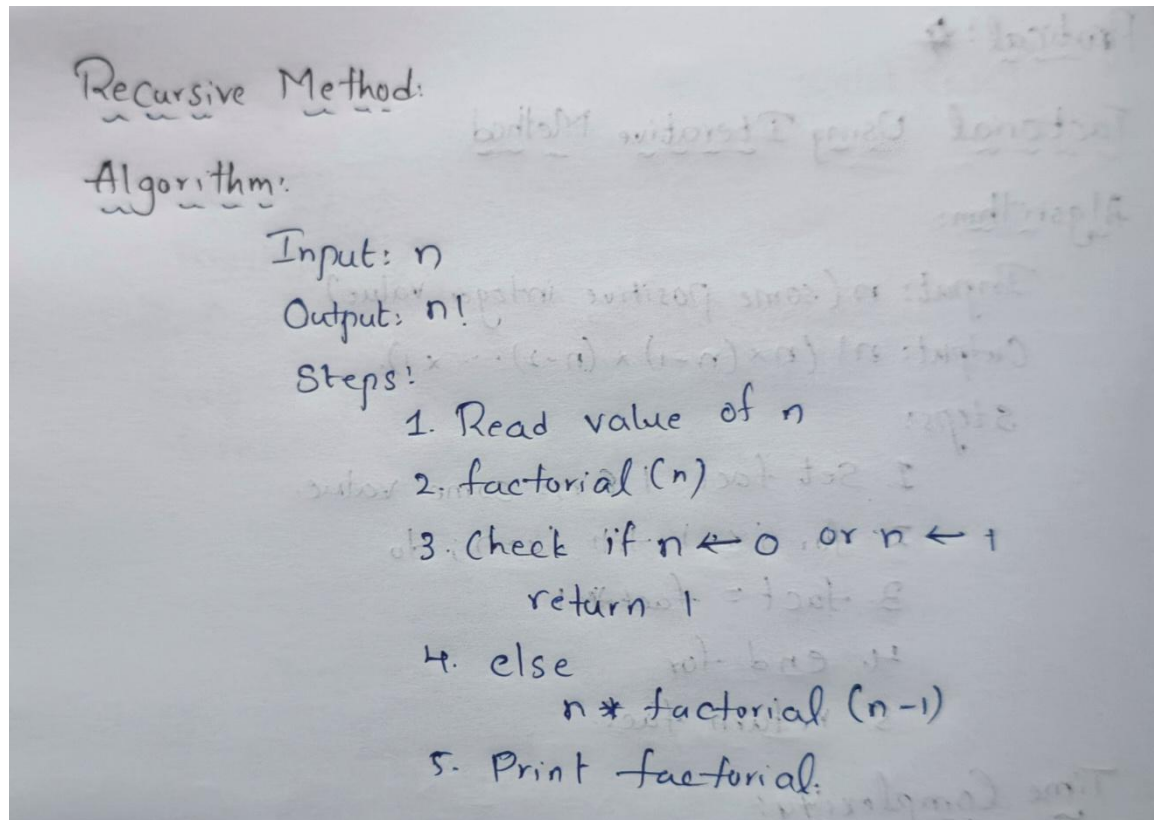
$T(n) = n \times 1$
 $T(n) = O(n)$

The loop always performs approximately n iterations regarding of input characteristics.

- Best Case = $O(n)$
- Average Case = $O(n)$
- Worst Case = $O(n)$



Recursive Algorithm:



Code:

```
#include<iostream>
using namespace std;

int factorial(int n)
{
    if(n==0 || n == 1)
    {
        return 1;
    }
    else
    {
        return n * factorial(n - 1);
    }
}

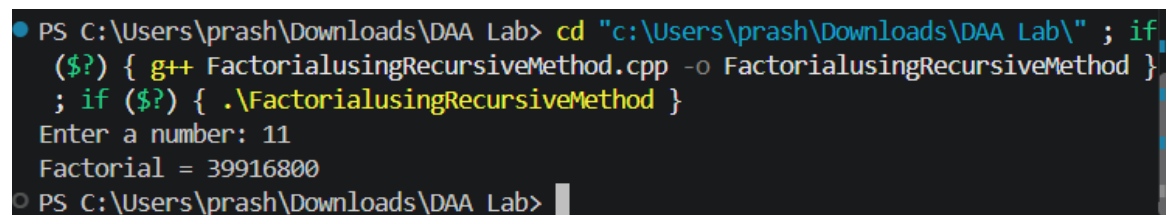
int main()
{
    int n;

    cout<<"Enter a value of n: ";
    cin >> n;
    int fact = factorial(n);

    cout << "Factorial of " << n << " = " << fact;

    return 0;
}
```

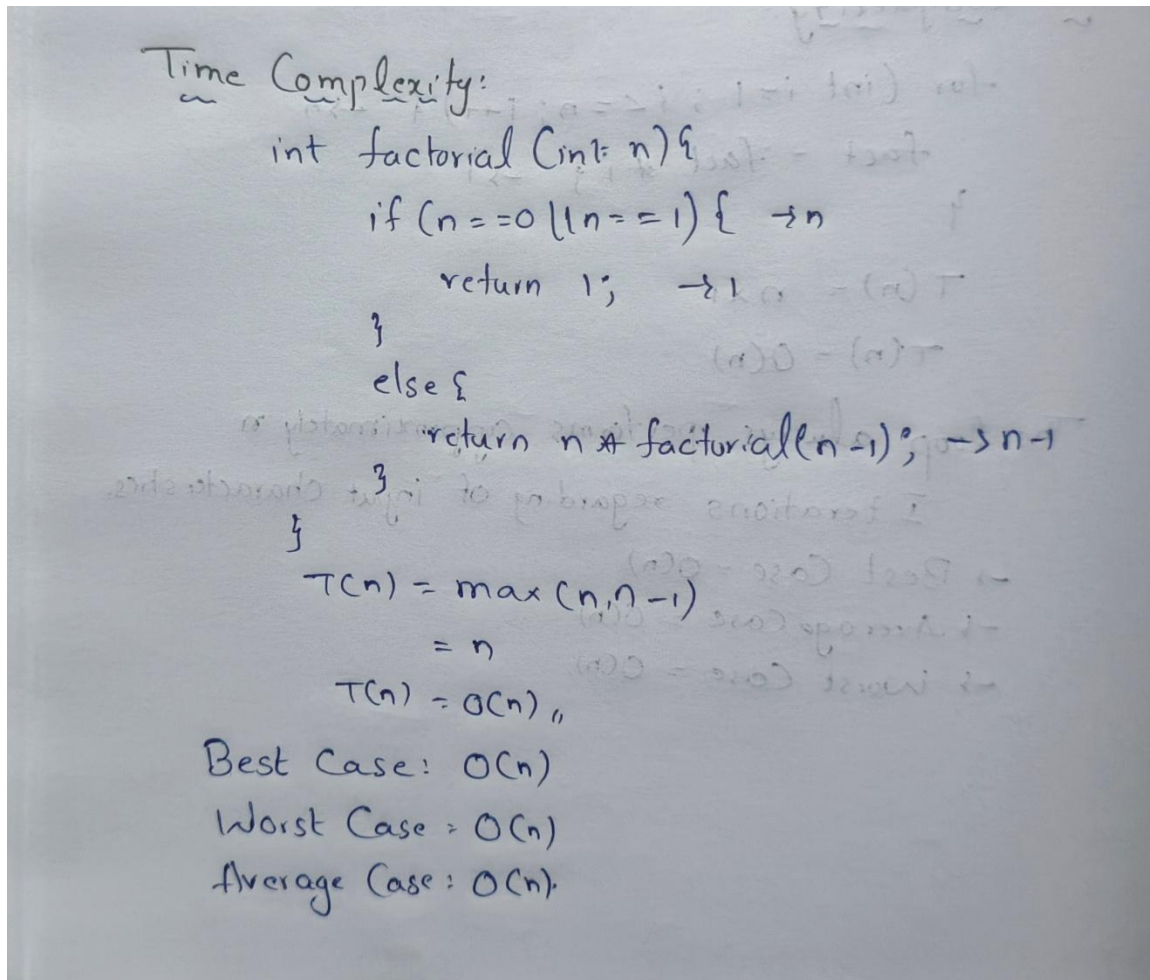
Output:



```
PS C:\Users\prash\Downloads\DAA Lab> cd "c:\Users\prash\Downloads\DAA Lab\" ; if ($?) { g++ FactorialusingRecursiveMethod.cpp -o FactorialusingRecursiveMethod } ; if ($?) { .\FactorialusingRecursiveMethod }
Enter a number: 11
Factorial = 39916800
PS C:\Users\prash\Downloads\DAA Lab>
```



Time complexity:



Conclusion:

Both iterative and recursive methods calculate the factorial correctly and have $O(n)$ time complexity. The iterative method uses a loop and generally requires less memory, while the recursive method uses function calls and requires $O(n)$ auxiliary space because of the recursion stack.